

MEMORIA TÉCNICA: LINGUACERT

Plataforma Descentralizada de Práctica Conversacional y Certificación Automatizada de Idiomas

Alumno	Asignatura	Institución
Fetta, Carolina	Teoría de la Computación II	Universidad Champagnat

1. Problema y Oportunidad

¿Qué problema resuelve el proyecto?

- **Falsificación de credenciales académicas:** las certificaciones de idiomas emitidas en formatos físicos o PDF tradicionales resultan vulnerables a la adulteración y al fraude documental, lo que obliga a los departamentos de RR.HH. a invertir tiempo y capital en procesos manuales de auditoría y validación de antecedentes.
- **Barrera económica en la práctica conversacional:** estudiantes y profesionales que buscan insertarse en mercados internacionales carecen de acceso económico a tutores nativos, limitando su entrenamiento a herramientas estáticas que no simulan un diálogo fluido en tiempo real.
- **Pérdida y burocracia de registros académicos:** la validación de un logro pasado depende de una entidad centralizada (universidad, academia) que puede tardar, cobrar o directamente desaparecer, dejando al estudiante sin forma de acreditar lo aprendido.
- **Validación del problema:** el problema se validó mediante el análisis de las ineficiencias actuales en los procesos de reclutamiento técnico y la creciente demanda de mecanismos de confianza "trustless" en aplicaciones web modernas, donde la identidad digital suele estar desligada de los logros verificables del usuario.

2. Solución Propuesta

LinguaCert es una Aplicación Descentralizada (DApp) que combina un tutor conversacional de Inteligencia Artificial con la inmutabilidad de la tecnología Blockchain para ofrecer un entorno de práctica y certificación de idiomas verificable de extremo a extremo.

Flujo general

El usuario conecta su identidad criptográfica (MetaMask). La plataforma genera, mediante IA, un tema de conversación adaptado al idioma y nivel elegidos. El alumno responde en el idioma objetivo y recibe, en cada turno, una devolución gramatical detallada junto a un puntaje de 0 a 100. Al alcanzar el umbral mínimo de aprobación en la cantidad de sesiones requeridas, el sistema habilita la emisión del certificado en la blockchain de Sepolia.

Diferenciación

A diferencia de las plataformas centralizadas tradicionales, LinguaCert no depende de una entidad emisora que pueda alterar los registros: la aprobación de las competencias lingüísticas se convierte en un activo digital, anclado a la dirección pública del estudiante y auditable públicamente en el explorador de bloques.

3. MVP Desarrollado

El Producto Mínimo Viable comprende un prototipo funcional de extremo a extremo, con las siguientes piezas efectivamente implementadas y verificadas en el repositorio:

- **Autenticación Web3:** inicio de sesión mediante la conexión de MetaMask vía window.ethereum, recuperando la dirección pública del usuario como identificador único de sesión.
- **Interfaz de chat guiada por IA:** conexión por turnos donde el usuario conversa en el idioma elegido y recibe, para cada mensaje, feedback (comentario gramatical en español), score (0-100) y corrected (versión corregida nativa), además de una réplica (reply) del tutor que continúa la charla.
- **Persistencia indexada por wallet:** el progreso (sesiones totales, sesiones aprobadas, último idioma/nivel) se guarda en localStorage bajo una clave derivada de la dirección de wallet (lib/progress.ts), separando el historial de cada usuario y persistiendo entre recargas del navegador.
- **Hash de integridad + subida a IPFS (ya implementado, no solo planificado):** al completar las sesiones requeridas, el frontend calcula un hash SHA-256 (computeSessionsHash) de todas las conversaciones aprobadas, sube ese detalle a IPFS mediante la API de Pinata (uploadToIPFS) y obtiene un CID (Content Identifier) verificable.
- **Emisión on-chain del certificado:** con el hash y el CID ya generados, el frontend invoca issueCertificate(...) del smart contract mediante ethers.js v6 (BrowserProvider + Signer), quedando el registro emitido on-chain junto al evento CertificateIssued.

4. Componentes Tecnológicos

Arquitectura general

La solución fue construida sobre Next.js 16 con Turbopack como motor de compilación, TypeScript para tipado estricto, y Tailwind CSS para la interfaz gamificada. El frontend usa componentes de cliente ("use client") para las interacciones con MetaMask y la conversación en tiempo real.

Inteligencia Artificial

El backend (app/api/chat/route.ts) consume el modelo google/gemini-2.5-flash mediante llamadas HTTP directas (fetch) al endpoint de chat completions de OpenRouter, sin pasar por un SDK intermedio. Se generan dos tipos de solicitudes: GET para proponer un tema de conversación nuevo (con una lista acotada de categorías para evitar temas repetitivos o poco naturales) y POST para evaluar cada mensaje del alumno, devolviendo feedback, score, corrected y reply en JSON estricto. Ante un error de la API externa, el backend aplica un mecanismo de contingencia (fallback) con contenido predefinido por idioma, para que el frontend nunca reciba una respuesta rota.

Blockchain y Smart Contract

Se desarrolló un contrato nativo en Solidity v0.8.19 (contracts/LinguaCert.sol), desplegado en la testnet de Ethereum Sepolia. El contrato mantiene un mapping(address => Certificate[]) que asocia a cada estudiante un arreglo de certificados, cada uno con idioma, nivel, cantidad de sesiones, hash de sesión (bytes32), CID de IPFS y timestamp. La función issueCertificate registra un nuevo certificado y emite el evento CertificateIssued, permitiendo su auditoría pública en Etherscan. Las funciones getCertificates y getCertificateCount permiten consultar el historial de una wallet.

Integración con Contenidos de la Materia

- **Lenguajes de Programación (Unidad I):** aplicación de ejecución asíncrona no bloqueante tanto en cliente como en servidor, a través de las rutas de API nativas de Next.js (NextRequest / NextResponse) para orquestar las llamadas externas a OpenRouter y Pinata.
- **Sistemas de Tipos (Unidad II):** tipado estricto con TypeScript en toda la base de código (lib/types.ts), con interfaces para las respuestas de la API de chat, los registros de sesión y el estado de progreso, validando la forma de los datos antes de incorporarlos al árbol de componentes de React.
- **Seguridad Informática e Introducción a Blockchain (Unidad IV):** uso de criptografía de clave pública y firmas digitales mediante el proveedor web3 (window.ethereum / ethers.js) para autenticación sin contraseñas, y despliegue de un contrato inteligente en la EVM (Sepolia) para sustituir el modelo de emisión centralizada por uno resistente a la censura y auditable.

5. Innovación e Impacto

- **Aspectos innovadores:** el proyecto introduce el concepto de "soberanía del logro académico": al descentralizar la emisión, el estudiante pasa a ser dueño de su historial de aprendizaje, verificable por cualquier tercero consultando el hash de la transacción en Etherscan, sin apostillas ni validaciones intermedias.
- **Impacto social y económico:** democratiza el acceso a la práctica conversacional de alto nivel, reduciendo a cero el costo de las tutorías continuas para estudiantes de entornos con recursos limitados que buscan certificar competencias bilingües para insertarse en el mercado laboral global.

6. Viabilidad y Sustentabilidad

- **Factibilidad técnica:** el proyecto es viable gracias a la madurez de los proveedores de infraestructura Web3 (nodos RPC, IPFS/Pinata) y a la disponibilidad de APIs de modelos de lenguaje de bajo costo y alta velocidad como Gemini 2.5 Flash.
- **Factibilidad económica:** al delegar el costo de almacenamiento (IPFS) y de gas de emisión en la propia red o en redes de segunda capa, la plataforma puede sostenerse sin depender de servidores centralizados de alto mantenimiento.
- **Estrategia de crecimiento:** modelo freemium con prácticas conversacionales diarias subsidiadas por cuotas gratuitas de APIs, y servicios premium opcionales para certificar niveles avanzados o acceder a tableros corporativos de reclutamiento.

7. Trabajo Realizado

- **Metodología empleada:** el equipo coordinó sus esfuerzos con un enfoque de hackathon y lineamientos ágiles, con un backlog inicial de tareas prioritarias para desarrollar un MVP funcional en un plazo de tres semanas.
- **Distribución de roles:** ingeniería de contratos inteligentes (Solidity / despliegue en Sepolia), desarrollo backend (rutas de API y prompts para la IA) y arquitectura frontend (maquetación y estados dinámicos del chat).

8. Aprendizajes y Propuestas Descartadas

- **Evolución del diseño:** inicialmente se planteó que la IA dictara el tema en texto libre, pero se descartó porque los modelos conversacionales tendían a generar saludos extensos o monólogos que rompían el contenedor

estético superior de la interfaz. Se iteró hacia un prompt estructurado que obliga a la IA a devolver JSON con topic, topicEs e intro bien diferenciados.

- **Lección técnica aprendida:** separar la generación del tema (GET) de la evaluación de cada mensaje (POST) permitió aislar fallos: si la API externa falla en un extremo, el fallback local evita que el usuario vea una pantalla rota, aunque pierda momentáneamente la personalización de la IA.
- **Trabajo pendiente identificado en esta revisión:** como próximos pasos concretos y verificables (no solo declarativos) quedan pendientes: migrar el contrato a un estándar ERC-721 para permitir transferencia y compatibilidad con marketplaces/wallets de NFT; mover la clave de OpenRouter a variables de entorno; y remover o documentar el módulo lib/gemini.ts, hoy sin uso.

9. Fuentes y Referencias

- Documentación oficial de Next.js y especificaciones de compilación de Turbopack.
- Especificaciones de TypeScript de Microsoft para el diseño de sistemas de tipos seguros.
- Documentación de Ethers.js v6 para la integración de proveedores y firmantes Web3.
- Documentación de referencia de la API de OpenRouter y especificaciones del modelo Google Gemini 2.5 Flash.
- Documentación de Pinata para el pineo de contenido JSON en IPFS.

10. Repositorio del Proyecto

<https://github.com/gcfetta/linguacert.git>